*Technical Appendix for*

# iGrow: A Smart Agriculture Solution to Autonomous Greenhouse Control

## Contents

**1. Supplementary Methods Description** ..... 1
1.1 Soft Actor-Critic ..... 1
1.2 Elitist Genetic Algorithm ..... 1
**2. Supplementary Hardware, Software, Parameters for Methods** ..... 2
2.1 Training NN-based simulator ..... 2
2.2 Training SAC algorithm ..... 2
2.3 Training EGA algorithm ..... 3
**3. Supplementary Deployment Details for Pilot Projects in the Real Autonomous Greenhouses** ..... 3
**References for Technical Appendix** ..... 3

# 1. Supplementary Methods Description

## 1.1 Soft Actor-Critic

In this work, we adopt Soft Actor-Critic (SAC) [1], a state-of-the-art off-policy reinforcement-learning algorithm, to optimize the AGC strategy. SAC interacts with the simulator constantly to collect transition tuples and store them. SAC alternates between a soft policy evaluation and a soft policy improvement. During soft policy evaluation, an NN-based soft Q-function is updated by minimizing a soft Bellman residual. During soft policy improvement, the policy is optimized by maximizing an entropy objective. The optimization process is given in Algorithm T1.

**Algorithm T1:** Soft Actor-Critic

**Input:** policy $\pi_\phi$, Q-function $Q_\theta$ and replay buffer $\mathcal{D}$
**Output:** Optimal $strategy$ after $N$ epochs

**for** $N_{epoch}$ *iterations* **do**
  **for** *each step* $t$ **do**
    $\mathbf{a}_t \leftarrow \pi_\phi(\mathbf{s}_t)$
    $\mathbf{s}_{t+1}, \mathbf{r}_{t+1} \leftarrow \text{Simulation}(\mathbf{s}_t, \mathbf{a}_t)$
    $\mathcal{D} \leftarrow \mathcal{D} \cup (\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})$
  **for** *each gradient step* **do**
    Sample random minibatch $\mathcal{B} \sim \mathcal{D}$
    $\mathcal{L}_e \leftarrow \text{BellmanResidual}(\mathcal{B})$
    $Q_\theta \leftarrow \text{Minimize}(\mathcal{L}_e)$
    $\mathcal{L}_i \leftarrow \text{KL-divergence}(\mathcal{B})$
    $\pi_\phi \leftarrow \text{Minimize}(\mathcal{L}_i)$
**return** $strategy$ $\pi_\phi$

## 1.2 Elitist Genetic Algorithm

Classical heuristic algorithms, such as the genetic algorithm, simulate biological evolution to solve deterministic optimization problems [2]. In this paper, we apply an elitist genetic algorithm (EGA) [3], which can accelerate the convergence of the AGC strategy.

Because the state space of each variable is different, we adopt multiple chromosomes to encode these variables. That is, in one cycle, the same variables share the same chromosomes by concatenation. Additionally, we use binary coding to represent strategies in the evolutionary process. During the evaluation, each individual belonging to the population will be decoded as the input of the simulation model to a decimal numeral. Simulation results will be taken as the objective values of each individual. The optimization procedure is summarized in Algorithm T2.

**Algorithm T2 :** Elitist Genetic Algorithm

**Input:** Initial population size $p_size$, iteration limit $itr_{limit}$, and the composition of polysomy $polysomy$
**Output:** Optimal $strategy$ after $itr_{limit}$ iterations

$pop \leftarrow$ InitializePopulation$(p_size)$
$strategy \leftarrow$ DecodeGenetics$(pop)$
$ObjV \leftarrow$ Simulation$(strategy)$
$pop.FitnV \leftarrow$ FitnessEvaluation$(pop, ObjV)$
$parent \leftarrow pop$
**for** $itr_{limit}$ *iterations* **do**
  $offspring \leftarrow$ RouletteSelect$(parent, p_size)$
  **for** *each* $polysomy\ c_i$ **do**
    $offspring \leftarrow$ Crossover$(offspring, c_i)$
    $offspring \leftarrow$ Mutation$(offspring, c_i)$
  $pop \leftarrow parent \cup offspring$
  $strategy \leftarrow$ DecodeGenetics$(pop)$
  $ObjV \leftarrow$ Simulation$(strategy)$
  $pop.FitnV \leftarrow$ FitnessEvaluation$(pop, Objv)$
  $parent \leftarrow$ SelectTopSizeFit$(pop, p_size)$
$BestIndividual \leftarrow$ SelectBestFit$(parent)$
$strategy \leftarrow$ DecodeGenetics$(BestIndividual)$
**return** $strategy$

In the actual planting process, we take the final accumulated net profit as the objective. Therefore, we have to optimize the AGC strategy throughout the whole planting period. The strategy requires the weather forecast model $p(w \mid t)$, which makes predictions for future weather to support optimization.

## 2. Supplementary Hardware, Software, Parameters for Methods

### 2.1 Training NN-based simulator

The hardware configuration: Intel (R) Core (TM) i7-9700K, one GeForce RTX 2080 Ti GPU, and the computing memory is 12 GB. We implement the model in a Python 3.6.13, PyTorch 1.7.1 environment.

(1) For the greenhouse climate module, the number of layers of the neural network is 5, and the number of neurons in each layer is 300.
(2) For the crop growth module, the number of layers of the neural network is 5, and the number of neurons in each layer is 300.
(3) For the production module, the number of layers of the neural network is 5, and the number of neurons in each layer is 600.

In the training phase of all the above neural networks, the batch size is 512, the learning rate is set to be 1e-4, and the number of the epoch is 300.

### 2.2 Training SAC algorithm

The hardware configuration: 8 GPU GeForce Titan X, and the computing memory is 60 GB. We implement the model in a Python 3.6.13, PyTorch 1.7.1 environment.

In the training phase of the SAC algorithm, the number of the epoch is 8000, the start step is 2000, and saving latest model per 50 epoch.

### 2.3 Training EGA algorithm

The hardware configuration: 8 GPU GeForce Titan X, and the computing memory is 60 GB. We implement the model in a Python 3.6.13, PyTorch 1.7.1 environment.
In the training phase of the EGA algorithm, the iteration of the population is 50000, population size is 6.

## 3. Supplementary Deployment Details for Pilot Projects in the Real Autonomous Greenhouses

The growers have to determine setpoints in advance, which will be sent to the cloud platform through an application programming interface (API) (the details of the controlled variables are shown in Supplementary Table S2). The cloud is responsible for generating instructions for the actuators according to the currently measured data of the greenhouse by sending setpoints every 10 minutes.

As shown in Supplementary Figure S2, the deployed sensors continuously measure the data with a frequency between every 1-3 seconds. Once the measured data is updated, it will be transmitted to the cloud platform via an embedded sensor system. Cloud-computing technology provides data cleaning, visualization, and weather forecasts. Growers can monitor the entire process in real-time through the graphical interface (see in Supplementary Figure S3). Also, the measured data can be downloaded from cloud storage.

## References for Technical Appendix

[1] F. Fu, Y. Kang, Z. Zhang, F. R. Yu and T. Wu, "Soft Actor–Critic DRL for Live Transcoding and Streaming in Vehicular Fog-Computing-Enabled IoV," in IEEE Internet of Things Journal, vol. 8, no. 3, pp. 1308-1321, 1 Feb.1, 2021, doi: 10.1109/JIOT.2020.3003398.
[2] Ponsich, Antonin, et al. "Constraint handling strategies in Genetic Algorithms application to optimal batch plant design." Chemical Engineering and Processing: Process Intensification 47.3 (2008): 420-434.
[3] Rani, Shweta, Bharti Suri, and Rinkaj Goyal. "On the effectiveness of using elitist genetic algorithm in mutation testing." Symmetry 11.9 (2019): 1145.